

# Sprint 2 Step 5: OpenAPI Documentation Enhancement

---

**Status:** ☒ Complete

**Validation:** 11/11 checkpoints passing

**Date:** 2026-08-29

## Overview

Enhanced OpenAPI/Swagger documentation for all API endpoints and models. All request/response models now include detailed Field() descriptors, examples, and documentation strings accessible through Swagger UI.

## What Was Enhanced

### 1. Model Documentation (Pydantic)

#### ChatRequest Model

- Added Field() descriptors for each parameter
- Included min\_length, max\_length constraints
- Added description and example for each field
- Supports query examples in Swagger UI

```
class ChatRequest(BaseModel):
    user_id: str = Field(
        ...,
        description="Unique identifier for the user",
        example="user@company.com"
    )
    message: str = Field(
        ...,
        min_length=1,
        max_length=2000,
        description="User message to send to the agent",
        example="What are the current sales trends?"
    )
    session_id: Optional[str] = Field(
        None,
        description="Optional session ID to continue conversation",
        example="session_123abc"
    )
```

#### ChatResponse Model

- Complete response examples
- Success case with generated\_response and session\_id
- Includes metadata like response\_id, timestamp
- Shows structure in Swagger UI

```
class ChatResponse(BaseModel):
    generated_response: str = Field(
        ...,
        description="Response generated by the agent",
        example="Based on the sales data..."
    )
    session_id: str = Field(
        ...,
        description="Session ID for future requests",
        example="session_123abc"
    )
    response_id: str = Field(
        ...,
        description="Unique identifier for this response",
        example="resp_456def"
    )
```

## ErrorResponse Model

- Documented all 12 error codes
- Field descriptions for error\_code, message, details
- Status code mapping (400, 401, 500, etc.)

```
class ErrorResponse(BaseModel):
    error_code: str = Field(
        ...,
        description="Machine-readable error code (e.g., MISSING_API_KEY, VERTEX_ERROR, etc.)",
        example="MISSING_API_KEY"
    )
    message: str = Field(
        ...,
        description="Human-readable error message",
        example="Authorization header or X-API-Key header required"
    )
    details: Optional[str] = Field(
        None,
        description="Additional error context or debugging information",
        example="Expected 'Authorization: Bearer <token>' or 'X-API-Key: <key>'"
    )
```

## HealthResponse Model

- Status field with values: "healthy", "degraded", "unhealthy"
- Includes version, uptime\_seconds, timestamp
- Environment information (GCP\_PROJECT\_ID, VERTEX\_ENGINE\_ID)

## 2. Endpoint Documentation

### GET /health

Enhanced documentation including:

- Purpose: Check API health status
- Response codes: 200 (healthy), 503 (degraded/unhealthy)
- Complete HealthResponse model example
- Status explanations for each field

```
GET /health
├─ Purpose: Health check endpoint for monitoring/load balancers
├─ Authentication: Not required
├─ Response: HealthResponse (200 OK)
└─ Example: {"status": "healthy", "version": "1.0.0", ...}
```

### POST /chat

Enhanced documentation including:

- Purpose: Send message to agent and get response
- Authentication: Required (API Key or OAuth2)
- Required headers: X-API-Key or Authorization
- Request body: ChatRequest model with example
- Response codes with examples:
  - 200 OK → ChatResponse
  - 400 Bad Request → ValidationError/AuthenticationError
  - 401 Unauthorized → MissingAPIKey error
  - 500 Internal Server Error → VertexError/ServerError

## 3. App-Level Documentation

Enhanced `app.description` (50+ line markdown):

```
# IsPilot API

REST API for enterprise agent orchestration through Vertex AI.

## Features
- **Multi-turn conversations**: Stateful sessions with Firestore
```

```
- **Enterprise authentication**: API Key or OAuth2
- **Cloud logging**: All requests logged to Google Cloud Logging
- **Error tracking**: 12 standardized error codes with context
- **Monitoring**: Cloud Monitoring metrics for performance tracking

## Authentication
- API Key: `X-API-Key: <key>` header
- OAuth2: `Authorization: Bearer <token>` header
- Required for POST /chat endpoint

## Error Codes
All errors return 12 standardized codes:
- MISSING_API_KEY, INVALID_API_KEY, AUTH_FAILED
- SERVER_CONFIG_ERROR, VERTEX_TIMEOUT, VERTEX_ERROR
- FIRESTORE_UNAVAILABLE, SESSION_NOT_FOUND
- VALIDATION_ERROR, NOT_FOUND, INTERNAL_SERVER_ERROR, RATE_LIMIT_EXCEEDED

## Headers
- X-Request-ID: Unique request identifier (auto-generated)
- X-API-Key: API key for authentication
- X-User-ID: User identifier for audit logging
- X-Session-ID: Session identifier for conversation context
```

## Files Modified

### [app/main.py](#)

- Updated `app.description` with comprehensive documentation
- Added exception handlers with error response documentation
- Configured OpenAPI tags and servers

### [app/models/responses.py](#)

- ChatRequest: Added Field() descriptors with constraints and examples
- ChatResponse: Added response examples and metadata fields
- ErrorResponse: Documented all error codes and structure
- HealthResponse: Added status explanations and field descriptions

### [app/models/errors.py](#)

- Documented all 12 error codes with descriptions
- Added HTTP status codes for each error type
- Added recovery suggestions for each error

### [app/api/chat.py](#)

- Added OpenAPI docstring to `/chat` endpoint
- Documented request/response examples
- Added response code explanations

# OpenAPI Output

The OpenAPI schema is automatically generated and available at:

- [GET /openapi.json](#) → Full OpenAPI 3.0 schema
- [GET /docs](#) → Swagger UI (interactive documentation)
- [GET /redoc](#) → ReDoc (alternative documentation)

## Swagger UI Features Now Available

### ☒ Request/Response Models

- Model schema visualization
- Field type validation rules
- Example values for copy-paste

### ☒ Endpoint Documentation

- Method and path
- Parameter descriptions
- Request/response examples
- HTTP status codes with explanations

### ☒ Try It Out

- Send actual requests from documentation
- See live responses
- Test error cases

### ☒ Error Documentation

- All 12 error codes documented
- Example error responses shown
- Recovery steps explained

## Validation Checkpoints

All 11 checkpoints verified:

1. ☒ ChatRequest Field descriptors present
2. ☒ ChatResponse examples included
3. ☒ ErrorResponse documents all 12 error codes
4. ☒ HealthResponse model documented
5. ☒ App description is comprehensive (50+ lines)
6. ☒ GET /health endpoint documented
7. ☒ POST /chat endpoint documented
8. ☒ Response codes with examples (200, 400, 401, 500)
9. ☒ OpenAPI schema available at /openapi.json
10. ☒ Swagger UI renders all descriptions
11. ☒ Model examples visible in Swagger UI

# Testing

Validation test: `test_step5_openapi.py` (deleted after passing all checkpoints)

```
# Run test to verify all documentation
python test_step5_openapi.py

# Expected output: ☒ ALL 11 CHECKPOINTS PASSED
```

## Usage

### For API Consumers

1. Visit Swagger UI: <https://ispilot-api-{hash}.run.app/docs>
2. Browse available endpoints
3. Click "Try it out" to test endpoints
4. Copy example requests for your application

### For Developers

1. OpenAPI schema at `/openapi.json`
2. Use schema in code generation tools
3. Pydantic models provide runtime validation
4. Field constraints prevent invalid requests

### For Integrations

1. Use OpenAPI schema for SDK generation
2. All field constraints enforced by Pydantic
3. Error codes standardized and documented
4. Examples aid integration development

## Integration with Error Handling

Each error response follows standard format:

```
{
  "error_code": "MISSING_API_KEY",
  "message": "Authorization header or X-API-Key header required",
  "details": "Expected 'Authorization: Bearer <token>' or 'X-API-Key: <key>'"
}
```

HTTP status code matches error severity:

- 400: Client error (validation, bad request)
- 401: Authentication error (missing/invalid credentials)

- 500: Server error (Vertex AI, infrastructure)

## Next Steps

- ☒ Completed in Sprint 2 Step 5
- Deploy to staging and validate documentation in live environment
- Use OpenAPI schema for client SDK generation
- Integrate error codes into API client libraries
- Monitor API usage patterns through logs and metrics

## Git History

**Commit:** "Sprint 2 Step 5: OpenAPI Documentation Enhancement with detailed endpoint and model descriptions"

See [SPRINT2\\_COMPLETION.md](#) for full Sprint 2 summary.